UTN
Facultad Regional Córdoba

TUP

Tecnicatura Universitaria en Programación

BASES DE DATOS II

Unidad Temática N°3:
Bases de Datos Orientada a Grafos

Guía
2° Año – 3° Cuatrimestre

CC BY ND NC

Índice

Bases de datos basadas en grafos 3

Problema 3.1: Introducción 3

Problema 3.2: Red Profesional de Desarrolladores 4

Problema 3.3:Trazabilidad de Lotes en una Cadena de Producción de Medicamentos 7

Problema 3.4: Optimización de Entregas en una Red de Logística Urbana 11

Bases de datos basadas en grafos

Problema 3.1: Introducción

Operaciones en Neo4j

Neo4j permite realizar una variedad de operaciones sobre los grafos, facilitando la creación, consulta, actualización y eliminación de datos mediante su lenguaje de consultas, Cypher.

Creación de nodos y relaciones

Para agregar nodos y establecer relaciones en Neo4j, se utilizan las siguientes sentencias:

```
CREATE (a:Persona {nombre: 'Ana', edad: 30})
CREATE (b:Persona {nombre: 'Carlos', edad: 35})
CREATE (a)-[:AMIGO_DE]->(b);
```

En este ejemplo:

Se crean dos nodos **Persona** con sus respectivas propiedades.

Se establece una relación **AMIGO_DE** entre ambos nodos.

Consultas de datos

Para recuperar información de la base de datos de grafos, se pueden ejecutar consultas con patrones de relaciones:

```
MATCH (p:Persona) RETURN p;
```

Este comando recupera todos los nodos de tipo Persona.

Para encontrar relaciones específicas:

```
MATCH (a:Persona)-[:AMIGO_DE]->(b:Persona) RETURN a, b;
```

Esta consulta devuelve todas las conexiones de amistad entre personas.

Actualización de datos

Los datos almacenados en Neo4j pueden ser modificados utilizando la sentencia **SET**:

```
MATCH (p:Persona {nombre: 'Ana'})
SET p.edad = 31;
```

Este comando actualiza la edad del **nodo Ana** a 31 años.

Eliminación de nodos y relaciones

Para eliminar un nodo junto con sus relaciones:

```
MATCH (p:Persona {nombre: 'Carlos'})
DETACH DELETE p;
```

Esto elimina el nodo Carlos y todas sus conexiones.

Problema 3.2: Red Profesional de Desarrolladores

Modelo

- (:Persona {nombre, edad, stack})
- (:Empresa {nombre, rubro})
- (:Proyecto {nombre, tecnologia})
- Relaciones:
  - (:Persona)-[:TRABAJA_EN]->(:Empresa)
  - (:Persona)-[:PARTICIPA_EN]->(:Proyecto)
  - (:Persona)-[:CONOCE]->(:Persona)

1. Crear tres personas:

- Ana, 30 años, stack: "Fullstack"
- Bruno, 28 años, stack: "Backend"
- Carla, 35 años, stack: "Frontend"

Resolución:

```
CREATE (:Persona {nombre: "Ana", edad: 30, stack: "Fullstack"});
CREATE (:Persona {nombre: "Bruno", edad: 28, stack: "Backend"});
CREATE (:Persona {nombre: "Carla", edad: 35, stack: "Frontend"});
```

2. Crear dos empresas:

- DevHouse, rubro: "Software"
- DataSolve, rubro: "Fintech"

```
Resolución:
CREATE (:Empresa {nombre: "DevHouse", rubro: "Software"});
CREATE (:Empresa {nombre: "DataSolve", rubro: "Fintech"});
```

3. Crear relaciones de trabajo:

- Ana y Bruno trabajan en DevHouse
- Carla trabaja en DataSolve

Resolución:

```
MATCH (a:Persona {nombre: "Ana"}), (e:Empresa {nombre: "DevHouse"})
CREATE (a)-[:TRABAJA_EN]->(e);

MATCH (b:Persona {nombre: "Bruno"}), (e:Empresa {nombre: "DevHouse"})
CREATE (b)-[:TRABAJA_EN]->(e);

MATCH (c:Persona {nombre: "Carla"}), (e:Empresa {nombre: "DataSolve"})
CREATE (c)-[:TRABAJA_EN]->(e);
```

---

4. Obtener los nombres de todas las personas

Resolución:

```
MATCH (p:Persona)
RETURN p.nombre;
```

5. Obtener los nombres de las personas que trabajan en DevHouse

Resolución:

```
MATCH (p:Persona)-[:TRABAJA_EN]->(e:Empresa {nombre: "DevHouse"})
RETURN p.nombre;
```

6. Obtener todas las personas y sus empresas

Resolución:

```
MATCH (p:Persona)-[:TRABAJA_EN]->(e:Empresa)
RETURN p.nombre AS Persona, e.nombre AS Empresa;
```

7. Crear proyectos y relaciones de participación

- Proyecto1: “Sistema de Gestión”, tecnología: Java
- Proyecto2: “Web Corporativa”, tecnología: React
- Ana y Carla participan en Proyecto2
- Bruno participa en Proyecto1

Resolución:

```
CREATE (:Proyecto {nombre: "Sistema de Gestión", tecnologia: "Java"});
CREATE (:Proyecto {nombre: "Web Corporativa", tecnologia: "React"});

MATCH (a:Persona {nombre: "Ana"}), (p:Proyecto {nombre: "Web Corporativa"})
CREATE (a)-[:PARTICIPA_EN]->(p);

MATCH (c:Persona {nombre: "Carla"}), (p:Proyecto {nombre: "Web Corporativa"})
CREATE (c)-[:PARTICIPA_EN]->(p);

MATCH (b:Persona {nombre: "Bruno"}), (p:Proyecto {nombre: "Sistema de Gestión"})
CREATE (b)-[:PARTICIPA_EN]->(p);
```

8. ¿Qué personas trabajan en proyectos con tecnología "React"?

Resolución:

```
MATCH (p:Persona)-[:PARTICIPA_EN]->(proj:Proyecto {tecnologia: "React"})
RETURN p.nombre;
```

9. Crear relaciones de conocimiento entre personas:

- Ana conoce a Bruno
- Bruno conoce a Carla

Resolución:

```
MATCH (a:Persona {nombre: "Ana"}), (b:Persona {nombre: "Bruno"})
CREATE (a)-[:CONOCE]->(b);

MATCH (b:Persona {nombre: "Bruno"}), (c:Persona {nombre: "Carla"})
CREATE (b)-[:CONOCE]->(c);
```

10. ¿Qué personas conoce Ana (directamente)?

Resolución:

```
MATCH (a:Persona {nombre: "Ana"})-[:CONOCE]->(conocido:Persona)
RETURN conocido.nombre;
```

11. ¿A quién puede llegar Ana en máximo 2 pasos?

Resolución:

```
MATCH (a:Persona {nombre: "Ana"})-[:CONOCE*1..2]->(alguien:Persona)
RETURN DISTINCT alguien.nombre;
```

12. ¿Qué pares de personas trabajan en la misma empresa?

Resolución:

```
MATCH (p1:Persona)-[:TRABAJA_EN]->(e:Empresa)<-[:TRABAJA_EN]-(p2:Persona)
WHERE p1.nombre < p2.nombre
RETURN p1.nombre, p2.nombre, e.nombre AS Empresa;
```

13. Borrar todos los nodos y relaciones (para reiniciar)

```
MATCH (n)
DETACH DELETE n;
```

Problema 3.3:Trazabilidad de Lotes en una Cadena de Producción de Medicamentos

Una empresa farmacéutica necesita asegurar la trazabilidad completa de sus medicamentos, desde la producción de insumos hasta la distribución final a farmacias. Debe poder responder a preguntas como:

- ¿Qué lotes están afectados si un insumo sale defectuoso?

- ¿Qué proveedores intervinieron en un medicamento específico?
- ¿Quién manipuló cada lote en la cadena?

El equipo de ingeniería de datos de la empresa decidió implementar una base de datos orientada a grafos con Neo4j, para modelar esta compleja red de relaciones. El gerente pide desarrollar un modelo funcional y ejecutar consultas que permitan responder a los requerimientos del área de control de calidad.

Objetivo del trabajo

Modelar y consultar una base de datos en Neo4j que permita representar de manera flexible y consultable la trazabilidad de insumos, procesos, operadores y productos finales.

Supuestos del sistema

1. Cada medicamento se produce a partir de **uno o más insumos**.
2. Los insumos pueden ser provistos por **distintos proveedores**.
3. Cada etapa del proceso productivo está a cargo de un **operador técnico**.
4. Un lote final puede ser distribuido a **varias farmacias**.
5. Si un insumo sale defectuoso, se debe poder trazar **todos los lotes afectados**.

Entidades sugeridas

Nodos:

- (:Lote {codigo, tipo, fechaProduccion}) → puede ser insumo o medicamento
- (:Proveedor {nombre, pais})
- (:Operador {nombre, especialidad})
- (:Farmacia {nombre, ciudad})

Relaciones:

- (:Proveedor)-[:PROVEE]->(:Lote)
- (:Lote)-[:COMPONE]->(:Lote) → relación de trazabilidad: insumo → medicamento
- (:Operador)-[:MANIPULA]->(:Lote)
- (:Lote)-[:ENTREGADO_A]->(:Farmacia)

Desafío: ¿Cómo se debería modelar este sistema?

Actividades a realizar:

1. Diseñar en papel el grafo (entidades y relaciones con propiedades).
2. Insertar nodos y relaciones en Neo4j.
3. Ejecutar consultas que respondan a:
   - ¿Qué medicamentos contienen insumos de un proveedor determinado?
   - ¿Qué operadores intervinieron en la producción de un lote específico?
   - ¿Qué farmacias recibieron productos que contengan un insumo en particular?
   - Si un insumo resultó defectuoso, ¿qué farmacias deben ser alertadas?

Sugerencia de carga de datos y resolución

Crear proveedores

```
CREATE (:Proveedor {nombre: "Quimex", pais: "Argentina"});
CREATE (:Proveedor {nombre: "BioPharma", pais: "Brasil"});
```

Crear insumos

```
CREATE (:Lote {codigo: "INS-101", tipo: "insumo", fechaProduccion:
date("2024-01-10")});
CREATE (:Lote {codigo: "INS-202", tipo: "insumo", fechaProduccion:
date("2024-01-15")});
```

Relacionar proveedor e insumo

```
MATCH (p:Proveedor {nombre: "Quimex"}), (l:Lote {codigo: "INS-101"})
CREATE (p)-[:PROVEE]->(l);

MATCH (p:Proveedor {nombre: "BioPharma"}), (l:Lote {codigo: "INS-202"})
CREATE (p)-[:PROVEE]->(l);
```

Crear lote de medicamento compuesto por insumos

```
CREATE (:Lote {codigo: "MED-300", tipo: "medicamento", fechaProduccion:
date("2024-02-01")});

MATCH (ins1:Lote {codigo: "INS-101"}), (ins2:Lote {codigo: "INS-202"}),
(med:Lote {codigo: "MED-300"})
CREATE (ins1)-[:COMPONE]->(med),
       (ins2)-[:COMPONE]->(med);
```

Crear operadores

```
CREATE (:Operador {nombre: "Ana Martínez", especialidad: "Control de calidad"});
CREATE (:Operador {nombre: "Juan Pérez", especialidad: "Producción"});
```

Asociar operadores con lotes

```
MATCH (o:Operador {nombre: "Ana Martínez"}), (l:Lote {codigo: "INS-101"})
CREATE (o)-[:MANIPULA]->(l);

MATCH (o:Operador {nombre: "Juan Pérez"}), (l:Lote {codigo: "MED-300"})
CREATE (o)-[:MANIPULA]->(l);
```

Crear farmacias y distribuir medicamento

```
CREATE (:Farmacia {nombre: "Farmacia Central", ciudad: "Córdoba"});
CREATE (:Farmacia {nombre: "Botiquín Norte", ciudad: "Rosario"});

MATCH (l:Lote {codigo: "MED-300"}), (f1:Farmacia {nombre: "Farmacia Central"}), (f2:Farmacia {nombre: "Botiquín Norte"})
CREATE (l)-[:ENTREGADO_A]->(f1),
       (l)-[:ENTREGADO_A]->(f2);
```

Consultas clave

¿Qué medicamentos usan insumos de "Quimex"?

```
MATCH (prov:Proveedor {nombre: "Quimex"})-[:PROVEE]->(insumo)-[:COMPONE]->(med:Lote {tipo: "medicamento"})
RETURN DISTINCT med.codigo;
```

¿Qué operadores manipularon el medicamento MED-300 o sus insumos?

```
MATCH (op:Operador)-[:MANIPULA]->(lote)
WHERE lote.codigo = "MED-300" OR (lote)-[:COMPONE]->(:Lote {codigo: "MED-300"})
RETURN DISTINCT op.nombre, op.especialidad;
```

Si el insumo INS-101 está contaminado, ¿qué farmacias están en riesgo?

```
MATCH (insumo:Lote {codigo: "INS-101"})-[:COMPONE]->(med:Lote)-[:ENTREGADO_A]->(f:Farmacia)
```

```
RETURN DISTINCT f.nombre, f.ciudad;
```

¿Qué medicamentos se entregaron a la ciudad de Rosario?

```
MATCH (lote:Lote {tipo: "medicamento"})-[:ENTREGADO_A]->(f:Farmacia
{ciudad: "Rosario"})
RETURN lote.codigo, f.nombre;
```

Problema 3.4: Optimización de Entregas en una Red de Logística Urbana

Una startup de tecnología logística llamada UrbanoGo está desarrollando un sistema para optimizar rutas y gestionar entregas en ciudades con alta densidad de tráfico. Las entregas se realizan a través de centros de distribución, conductores y una red de puntos de entrega (clientes y lockers inteligentes).

El área de planificación quiere modelar toda esta red en una base de datos de grafos con Neo4j, de modo que sea fácil:

- Consultar la ruta de un paquete.
- Determinar los centros o rutas más congestionados.
- Identificar cuellos de botella.
- Visualizar relaciones entre pedidos, vehículos y zonas urbanas.

Objetivo del trabajo

Modelar y consultar una base de datos en Neo4j que permita representar el recorrido de paquetes, conductores, centros de distribución y zonas de entrega, facilitando la toma de decisiones logísticas.

Entidades sugeridas
Nodos:

- (:CentroDistribucion {nombre, ciudad})
- (:Zona {nombre})
- (:Cliente {nombre, direccion})
- (:Locker {id, ubicacion})
- (:Conductor {nombre, patente_vehiculo})
- (:Paquete {id, peso, estado})

Relaciones:

- (:CentroDistribucion)-[:CUBRE]->(:Zona)

- (:Conductor)-[:ASIGNADO_A]->(:CentroDistribucion)
- (:Conductor)-[:ENTREGA]->(:Paquete)
- (:Paquete)-[:DESTINADO_A]->(:Cliente|:Locker)
- (:Paquete)-[:PASO_POR]->(:CentroDistribucion)
- (:Zona)-[:TIENE]->(:Cliente|:Locker)

Escenario

Se debe responder a los siguientes requerimientos reales:

1. **Auditoría**: ¿Por qué centros de distribución pasó un paquete específico?
2. **Cobertura**: ¿Qué zonas atiende un centro?
3. **Carga de trabajo**: ¿Cuántos paquetes tiene asignado cada conductor?
4. **Destino**: ¿Cuáles son los paquetes dirigidos a lockers?
5. **Ruta completa**: ¿Cuál fue el recorrido de un paquete desde su centro inicial hasta el cliente?

Carga inicial de datos (resumen de creación de nodos y relaciones)

Centros, zonas y cobertura

```
CREATE (:CentroDistribucion {nombre: "CD-Norte", ciudad: "Córdoba"});
CREATE (:CentroDistribucion {nombre: "CD-Sur", ciudad: "Córdoba"});

CREATE (:Zona {nombre: "Alta Córdoba"});
CREATE (:Zona {nombre: "Nueva Córdoba"});

MATCH (cd:CentroDistribucion {nombre: "CD-Norte"}), (z:Zona {nombre: "Alta
Córdoba"})
CREATE (cd)-[:CUBRE]->(z);

MATCH (cd:CentroDistribucion {nombre: "CD-Sur"}), (z:Zona {nombre: "Nueva
Córdoba"})
CREATE (cd)-[:CUBRE]->(z);
```

Clientes y lockers

```
CREATE (:Cliente {nombre: "María López", direccion: "Sucre 1200"});
```

```
CREATE (:Cliente {nombre: "Pedro Ríos", direccion: "Obispo Trejo 800"});
CREATE (:Locker {id: "LK-001", ubicacion: "Plaza España"});

MATCH (z:Zona {nombre: "Nueva Córdoba"})
MATCH (c1:Cliente {nombre: "Pedro Ríos"}), (l:Locker {id: "LK-001"})
CREATE (z)-[:TIENE]->(c1), (z)-[:TIENE]->(l);
```

Conductores y asignación

```
CREATE (:Conductor {nombre: "Carlos Vega", patente_vehiculo: "AB123CD"});
CREATE (:Conductor {nombre: "Lucía Méndez", patente_vehiculo: "EF456GH"});

MATCH (c:Conductor {nombre: "Carlos Vega"}), (cd:CentroDistribucion
{nombre: "CD-Norte"})
CREATE (c)-[:ASIGNADO_A]->(cd);
```

Paquetes y su ruta

```
CREATE (:Paquete {id: "P-001", peso: 2.5, estado: "en tránsito"});
CREATE (:Paquete {id: "P-002", peso: 1.2, estado: "entregado"});

MATCH (p:Paquete {id: "P-001"}), (cd:CentroDistribucion {nombre: "CD-
Norte"})
CREATE (p)-[:PASO_POR]->(cd);

MATCH (p:Paquete {id: "P-002"}), (cd1:CentroDistribucion {nombre: "CD-
Norte"}), (cd2:CentroDistribucion {nombre: "CD-Sur"})
CREATE (p)-[:PASO_POR]->(cd1),
       (p)-[:PASO_POR]->(cd2);
```

Entrega a clientes y lockers

```
MATCH (p:Paquete {id: "P-001"}), (c:Cliente {nombre: "María López"})
CREATE (p)-[:DESTINADO_A]->(c);

MATCH (p:Paquete {id: "P-002"}), (l:Locker {id: "LK-001"})
CREATE (p)-[:DESTINADO_A]->(l);
```

Relación conductor-paquete

```
MATCH (p:Paquete {id: "P-002"}), (c:Conductor {nombre: "Carlos Vega"})
CREATE (c)-[:ENTREGA]->(p);
```

Consultas sugeridas (resueltas)

1. ¿Por qué centros de distribución pasó el paquete "P-002"?

```
MATCH (p:Paquete {id: "P-002"})-[:PASO_POR]->(cd:CentroDistribucion)
RETURN p.id, cd.nombre;
```

2. ¿Qué zonas atiende el centro "CD-Sur"?

```
MATCH (:CentroDistribucion {nombre: "CD-Sur"})-[:CUBRE]->(z:Zona)
RETURN z.nombre;
```

3. ¿Cuántos paquetes está entregando cada conductor?

```
MATCH (c:Conductor)-[:ENTREGA]->(p:Paquete)
RETURN c.nombre, count(p) AS paquetes_asignados;
```

4. ¿Qué paquetes fueron enviados a lockers?

```
MATCH (p:Paquete)-[:DESTINADO_A]->(l:Locker)
RETURN p.id, l.ubicacion;
```

5. ¿Cuál fue el recorrido del paquete "P-002" hasta su entrega?

```
MATCH (p:Paquete {id: "P-002"})-[:PASO_POR]->(cd:CentroDistribucion)
WITH p, collect(cd.nombre) AS centros
MATCH (p)-[:DESTINADO_A]->(dest)
RETURN p.id, centros, dest;
```

Extensiones posibles

- Incorporar estados temporales del paquete (:ACTUALIZADO_A) con fechas (como subgrafo temporal).
- Consultar la eficiencia por zona (cantidad de entregas exitosas por zona).
- Estimar rutas alternativas si un centro está inactivo.

Atribución-No Comercial-Sin Derivadas

Se permite descargar esta obra y compartirla, siempre y cuando no sea modificado y/o alterado su contenido, ni se comercialice. Referenciarlo de la siguiente manera: Universidad Tecnológica Nacional Facultad Regional Córdoba (S/D). Material para la Tecnicatura Universitaria en Programación, modalidad virtual, Córdoba, Argentina.